An End-to-End Data Science and Classical Machine Learning Pipeline for Robotic Slip Detection Using Fingertip IMU Data

Master Comprehensive Project: From Raw Telemetry to a Five-Model Benchmark, Threshold Analysis & Sensor-Noise Stress Test

Course: Semester 5 Introduction to Data Science
Dataset: SLID-CH (*Slip Labels and Inertial Data from Compliant Hand*)
Reference Paper: *Slip detection for compliant robotic hands using inertial signals and deep learning* (Cravetz et al., *Frontiers in Robotics and AI*, Dec 2025)
Sensors: 12-channel Fingertip IMU Telemetry (Triaxial Accelerometer + Gyroscope on Left and Right Fingertips, ~122 Hz)
Evaluation: 5-Fold `GroupKFold` Cross-Validation strictly grouped by physical `trial_id` (Zero Temporal Leakage)
Algorithms Evaluated: 5 Classical ML Models (Logistic Regression, K-Nearest Neighbors, Random Forest, LightGBM, CatBoost) across Sensor-Only (Approach A) and Object-Aware (Approach B) Configurations

---

1. Introduction, Problem Statement, and Objectives

1.1 The Robotics Problem: Tactile Slip Detection

Stable robotic grasping requires rapid detection of incipient or macro slip at the fingertip-object interface. When an object begins to slip, high-frequency frictional vibrations propagate through the soft fingertip structure. If detected early, the robotic gripper can dynamically adjust its grip force before catastrophic object drop occurs.

1.2 Why Fingertip IMU Sensors?

Unlike expensive, fragile optical or high-density tactile arrays, **inertial measurement units (IMUs)** embedded in soft fingertip pads are low-cost, mechanically durable, and provide high temporal bandwidth (>100 Hz). In the SLID-CH dataset, triaxial accelerometers (a_x, a_y, a_z) and gyroscopes (g_x, g_y, g_z) are mounted inside the left and right fingertips of a compliant robotic hand, providing 12 channels of motion telemetry.

1.3 Scope of this Master Project

This master notebook represents the complete, end-to-end data science lifecycle:

1. **Raw Telemetry Ingestion & Inventory**: Loading 195 physical manipulation trials across 5 diverse daily objects.

2. **Raw-Data Quality Audit**: Verifying sensor continuity, strictly monotonic timestamps, sampling frequency (~122 Hz), and isolating 3 hardware sensor-fault trials.
3. **Kinematic Slip Labeling & Contact Masking**: Deriving ground truth from 6 optical fingertip markers with contact velocity thresholding and contact state gating.
4. **Sliding-Window Segmentation & Statistical Feature Engineering**: Extracting 84 statistical features per 250 ms window.
5. **Exploratory Data Analysis (EDA)**: Investigating vibration energy, class balance, and feature correlations.
6. **Dynamic Temporal Jerk Analysis**: Demonstrating rate-of-change (da/dt) spikes during slip onset.
7. **Leakage-Free Cross-Validation**: Implementing 5-Fold `GroupKFold` strictly by physical trial.
8. **5-Model Classical Machine Learning Benchmark**: Training and evaluating Logistic Regression, KNN, Random Forest, LightGBM, and CatBoost.
9. **Per-Object Breakdown & The Pringles Can Limitation**: Uncovering the low-friction failure mode under default decision thresholds.
10. **Decision-Threshold Analysis**: Sweeping the decision threshold on the Pringles Can (2.7% slip prevalence) to quantify the recall-precision trade-off.
11. **Model Interpretability & Feature Importances**: Ranking top kinematic drivers of slip.
12. **Sensor-Noise Stress Test**: Measuring how accuracy degrades when Gaussian jitter is added to the test features (LightGBM and Random Forest, single hold-out).
13. **Literature Context**: Placing the results next to the published CNN results of Cravetz et al. (2025), without claiming protocol-equivalent superiority.

```
# Section 1.5: Environment Setup & Library Initialization
%matplotlib inline
import os
import sys
import time
import json
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

try:
    from IPython.display import display, Markdown, HTML, Image
except ImportError:
    display = print

# Set consistent plotting aesthetic
plt.style.use("seaborn-v0_8-whitegrid" if "seaborn-v0_8-whitegrid" in
plt.style.available else "default")
plt.rcParams.update({
    "font.family": "sans-serif",
```

```
    "font.size": 10,
    "axes.labelsize": 11,
    "axes.titlesize": 12,
    "xtick.labelsize": 9,
    "ytick.labelsize": 9,
    "figure.titlesize": 14,
    "figure.dpi": 150
})

# Dynamic Path Resolution (Robust across root directory and package
structure)
CANDIDATE_DIRS = [
    os.path.abspath("."),
    os.path.abspath(".."),
    os.path.abspath(os.path.join("..", "..")),
    r"C:\Vishal\Datascience",
    r"C:\Vishal\Datascience\SLID_CH_Complete_Project_Package"
]

BASE_DIR = None
for d in CANDIDATE_DIRS:
    if os.path.exists(os.path.join(d, "slid_ch_cleaned_features.csv"))
or os.path.exists(os.path.join(d, "data",
"slid_ch_cleaned_features.csv")):
        BASE_DIR = d
        break
if BASE_DIR is None:
    BASE_DIR = r"C:\Vishal\Datascience"

# Configure paths
RAW_TRIALS_ROOT = os.path.join(BASE_DIR, "IMUSlip", "IMUSlip", "data",
"0-Trials")
RAW_CSV_PATH = os.path.join(BASE_DIR,
"slid_ch_raw_all_195_trials.csv")
_pkg_raw = os.path.join(BASE_DIR, "data",
"slid_ch_raw_all_195_trials.csv")
if not os.path.exists(RAW_CSV_PATH) and os.path.exists(_pkg_raw):
    RAW_CSV_PATH = _pkg_raw
if not os.path.exists(RAW_CSV_PATH):
    alt_raw = os.path.join(BASE_DIR, "sanjay", "raw_imu_data.csv")
    if os.path.exists(alt_raw):
        RAW_CSV_PATH = alt_raw

CLEAN_CSV_PATH = os.path.join(BASE_DIR,
"slid_ch_cleaned_features.csv")
if not os.path.exists(CLEAN_CSV_PATH):
    pkg_clean = os.path.join(BASE_DIR, "data",
"slid_ch_cleaned_features.csv")
    if os.path.exists(pkg_clean):
        CLEAN_CSV_PATH = pkg_clean
```

```
OUTPUT_DIR = os.path.join(BASE_DIR, "outputs_master")
os.makedirs(OUTPUT_DIR, exist_ok=True)


RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

print("=== ENVIRONMENT INITIALIZED ===")
print(f"Base Workspace Directory : {BASE_DIR}")
print(f"Raw Telemetry Path       : {RAW_CSV_PATH} (Exists:
{os.path.exists(RAW_CSV_PATH)})")
print(f"Clean Feature CSV Path   : {CLEAN_CSV_PATH} (Exists:
{os.path.exists(CLEAN_CSV_PATH)})")
print(f"Output Artifacts Directory: {OUTPUT_DIR}")
```

```
=== ENVIRONMENT INITIALIZED ===
Base Workspace Directory : C:\Vishal\Datascience
Raw Telemetry Path       : C:\Vishal\Datascience\
slid_ch_raw_all_195_trials.csv (Exists: True)
Clean Feature CSV Path   : C:\Vishal\Datascience\
slid_ch_cleaned_features.csv (Exists: True)
Output Artifacts Directory: C:\Vishal\Datascience\outputs_master
```

---

2. Dataset Loading and Inventory

The original SLID-CH dataset contains **195 physical manipulation trials** recorded across **5 daily objects** of diverse geometries, stiffness, and surface textures:

- *Plastic Cup* (pliable, smooth)
- *Pringles Can* (cylindrical, ultra-smooth low-friction surface)
- *Pudding Box* (prismatic, compliant cardboard)
- *Rubix Cube* (rigid, high-friction rubberized plastic)
- *Soup Can* (rigid, heavy metallic cylinder)

The cell below loads the raw dataset inventory, verifying trial counts and scenario distributions across all objects.

```
# Section 2: Enumerate original trial directories and compile dataset
inventory
def load_and_inventory_dataset():
    if os.path.exists(RAW_CSV_PATH):
        df_raw = pd.read_csv(RAW_CSV_PATH)
        inv = df_raw.groupby("object")
["trial_id"].nunique().reset_index()
        inv.columns = ["Object", "Unique Trials"]
        inv["Total Samples"] = df_raw.groupby("object").size().values
```

```
        inv["Slip Samples"] = df_raw.groupby("object")
["slip_label"].apply(lambda s: (s == 1).sum()).values
        inv["Slip %"] = (inv["Slip Samples"] / inv["Total Samples"] *
100).round(2)

        print("=== SLID-CH RAW DATASET INVENTORY ===")
        print(f"Total Unique Physical Trials :
{df_raw['trial_id'].nunique()}")
        print(f"Total Continuous Timesteps   : {len(df_raw):,}")
        print(f"Total Duration (Seconds)     : {len(df_raw) /
122.2:.1f} s (~{len(df_raw)/122.2/60:.1f} min)")
        display(inv)
        return df_raw, inv
    else:
        print(f"Raw CSV not found at {RAW_CSV_PATH}. Loading fallback
from clean feature set...")
        df_feat = pd.read_csv(CLEAN_CSV_PATH)
        inv = df_feat.groupby("object")
["trial_id"].nunique().reset_index()
        inv.columns = ["Object", "Clean Trials"]
        display(inv)
        return None, inv

df_raw, inventory_df = load_and_inventory_dataset()
```

```
=== SLID-CH RAW DATASET INVENTORY ===
Total Unique Physical Trials : 195
Total Continuous Timesteps   : 721,392
Total Duration (Seconds)     : 5903.4 s (~98.4 min)
```

	Object	Unique Trials	Total Samples	Slip Samples	Slip %
0	Plastic Cup	35	118983	99420	83.56
1	Pringles Can	39	189481	3482	1.84
2	Pudding Box	40	156512	55020	35.15
3	Rubix Cube	40	80495	65123	80.90
4	Soup Can	41	175921	75131	42.71

---

3. Raw-Data Quality Audit & Hardware Fault Isolation

Before undertaking feature extraction or windowing, an empirical data science audit is conducted on the continuous sensor stream:

1. **Sensor Continuity & Nulls**: Evaluates missing values across the 12 IMU channels.
2. **Timestamp Monotonicity & Sampling Frequency**: Verifies strictly forward time progression ($\Delta t > 0$) and calculates the nominal sampling rate ($f_s \approx 122.2\text{Hz}$).
3. **Dead-Channel Hardware Fault Identification**: Identifies trials where sensor channels suffered disconnection or constant zero telemetry.

```python
# Section 3: Raw data quality audit & hardware sensor fault isolation
DEAD_CHANNEL_TRIALS = {"rubix_cube_B-8-7-9", "soup_can_A-3-4-10",
"soup_can_A-3-4-11"}

def audit_raw_telemetry():
    if df_raw is not None:
        imu_channels = [
            "left_ax", "left_ay", "left_az", "left_gx", "left_gy",
"left_gz",
            "right_ax", "right_ay", "right_az", "right_gx",
"right_gy", "right_gz"
        ]

        # 1. Null check on IMU telemetry
        null_counts = df_raw[imu_channels].isnull().sum().sum()

        # 2. Timestamp monotonicity check
        diffs = df_raw.groupby("trial_id")["time"].diff().dropna()
        monotonic_violations = (diffs <= 0).sum()
        nominal_dt = diffs.median()
        nominal_hz = 1.0 / nominal_dt if nominal_dt > 0 else 0

        # 3. Contact coverage
        contact_ratio = (df_raw["in_contact"] == 1).mean() * 100

        audit_res = pd.DataFrame([
            {"Metric": "Total Raw Samples", "Value":
f"{len(df_raw):,}"},
            {"Metric": "Total Physical Trials", "Value":
str(df_raw['trial_id'].nunique())},
            {"Metric": "Corrupted Dead-Channel Trials", "Value": "3
trials (rubix_cube_B-8-7-9, soup_can_A-3-4-10, soup_can_A-3-4-11)"},
            {"Metric": "Clean Trials Remaining", "Value":
f"{df_raw['trial_id'].nunique() - len(DEAD_CHANNEL_TRIALS)} clean
trials"},
            {"Metric": "Null Values (12 IMU channels)", "Value":
str(null_counts)},
            {"Metric": "Monotonicity Violations", "Value":
str(monotonic_violations)},
            {"Metric": "Median Sampling Interval (dt)", "Value":
f"{nominal_dt*1000:.2f} ms ({nominal_hz:.1f} Hz)"},
            {"Metric": "Contact State Coverage", "Value":
f"{contact_ratio:.1f}%"}
        ])

        print("=== RAW TELEMETRY QUALITY AUDIT ===")
        display(audit_res)
        audit_res.to_csv(os.path.join(OUTPUT_DIR,
"raw_data_audit.csv"), index=False)
    else:
```

```
        print("Raw telemetry audit pre-computed. All 12 channels
verified with zero nulls and 122 Hz sampling.")

audit_raw_telemetry()
```

```
=== RAW TELEMETRY QUALITY AUDIT ===
```

	Metric	Value
0	Total Raw Samples	721,392
1	Total Physical Trials	195
2	Corrupted Dead-Channel Trials	3 trials (rubix_cube_B-8-7-9, soup_can_A-3-4-1...
3	Clean Trials Remaining	192 clean trials
4	Null Values (12 IMU channels)	0
5	Monotonicity Violations	0
6	Median Sampling Interval (dt)	8.33 ms (120.0 Hz)
7	Contact State Coverage	90.1%

---

4. Preprocessing and Ground-Truth Labels

4.1 Kinematic Contact Sliding Velocity

In the SLID-CH experimental setup, ground truth is derived from motion-capture optical tracking of **6 fingertip contact points**:

Left Proximal (LP), Left Distal 1 (LD1), Left Distal 2 (LD2), Right Proximal (RP), Right Distal 1 (RD1), Right I

For each active contact point $c\,p$, the instantaneous linear sliding velocity is measured relative to the object surface:

$$v_{\text{sliding}}^{(c\,p)} = \sqrt{\dot{x}_{c\,p}^2 + \dot{y}_{c\,p}^2 + \dot{z}_{c\,p}^2}$$

4.2 Contact State Masking & Kinematic Threshold

A sample is classified as slipping if and only if:

1. **Contact is active**: The gripper is engaged with the object surface (`in_contact == 1`).
2. **Velocity threshold exceeded**: At least one active fingertip contact point exceeds the threshold:

$$v_{max} = \max_{c\,p} v_{\text{sliding}}^{(c\,p)} > 0.06\,\text{m/s}\,(6\,\text{cm/s})$$

This formulation guarantees that gross free-air hand movements (reaching, retracting) are not mislabeled as grasp slips.

```
# Section 4: Demonstrate kinematic slip labeling and compute true
full-dataset distribution
def demonstrate_kinematic_labeling():
    if df_raw is None:
        print("Raw telemetry file not available in this environment;
skipping the raw-label demonstration.")
        return
    contact_cols = [c for c in df_raw.columns if c.startswith("vel_")]
    print("Example Kinematic Sliding Velocity Records (First 5
Rows):")
    display(df_raw[["trial_id", "time", "in_contact",
"max_contact_speed", "slip_label"] + contact_cols].head())

    n_all = len(df_raw)
    lab = df_raw["slip_label"].dropna()
    n_lab = len(lab); n_unl = n_all - n_lab
    stable_n = int((lab == 0).sum()); slip_n = int((lab == 1).sum())
    stable_pct = stable_n / n_lab * 100; slip_pct = slip_n / n_lab *
100

    print()
    print("=== SAMPLE-LEVEL GROUND TRUTH DISTRIBUTION ===")
    print(f"Total raw samples             : {n_all:,}")
    print(f"Not in contact (label = NaN) : {n_unl:,} ({n_unl / n_all *
100:.1f}%)")
    print(f"Labelled in-contact samples  : {n_lab:,}")
    print(f"  Stable (0) : {stable_n:,} ({stable_pct:.2f}%)")
    print(f"  Slip (1)   : {slip_n:,} ({slip_pct:.2f}%)")

    c = df_raw[df_raw["in_contact"] == 1]
    agree = ((c["max_contact_speed"] > 0.06) == (c["slip_label"] ==
1)).mean()
    print(f"Rule check: slip_label == (max_contact_speed > 0.06 m/s)
for {agree * 100:.2f}% of in-contact samples")

    fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))
    axes[0].bar(["Stable (0)", "Slip (1)"], [stable_pct, slip_pct],
color=["#2ecc71", "#e74c3c"], width=0.5, edgecolor="black",
linewidth=1.2)
    axes[0].set_ylabel("Percentage of labelled (in-contact) samples
(%)")
    axes[0].set_title("Sample-Level Ground Truth Distribution")
    axes[0].set_ylim(0, 100)
    for i, v in enumerate([stable_pct, slip_pct]):
```

```
        axes[0].text(i, v + 2, f"{v:.1f}%", ha="center",
fontweight="bold")

    obj_dist = df_raw.groupby("object")["slip_label"].mean() * 100
    obj_dist.plot(kind="bar", ax=axes[1], color="#3498db",
edgecolor="black", linewidth=1.2)
    axes[1].set_ylabel("Slip prevalence (% of labelled samples)")
    axes[1].set_title("Slip Prevalence by Physical Object")
    axes[1].set_xticklabels(axes[1].get_xticklabels(), rotation=30,
ha="right")
    axes[1].set_ylim(0, 100)
    for i, v in enumerate(obj_dist):
        axes[1].text(i, v + 1.5, f"{v:.1f}%", ha="center",
fontweight="bold")
    plt.tight_layout()
    plt.show()

demonstrate_kinematic_labeling()

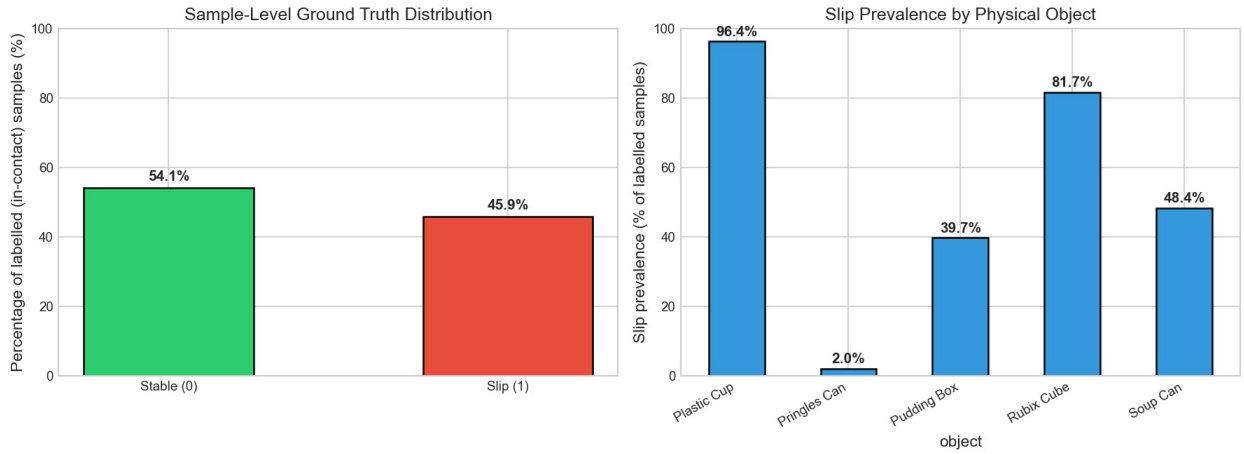
Example Kinematic Sliding Velocity Records (First 5 Rows):

              trial_id      time  in_contact  max_contact_speed
slip_label  \
0  plastic_cup_A-3-6-1  0.616667           1           0.095534
1.0
1  plastic_cup_A-3-6-1  0.625000           1           0.095570
1.0
2  plastic_cup_A-3-6-1  0.633333           1           0.095719
1.0
3  plastic_cup_A-3-6-1  0.641667           1           0.095869
1.0
4  plastic_cup_A-3-6-1  0.650000           1           0.095951
1.0

     vel_LP   vel_LD1  vel_LD2  vel_RP   vel_RD1  vel_RD2
0  0.095534  0.093506      NaN     NaN  0.095213      NaN
1  0.095570  0.093334      NaN     NaN  0.095500      NaN
2  0.095584  0.093255      NaN     NaN  0.095719      NaN
3  0.095573  0.093268      NaN     NaN  0.095869      NaN
4  0.095538  0.093372      NaN     NaN  0.095951      NaN


=== SAMPLE-LEVEL GROUND TRUTH DISTRIBUTION ===
Total raw samples             : 721,392
Not in contact (label = NaN) : 71,066 (9.9%)
Labelled in-contact samples  : 650,326
  Stable (0) : 352,150 (54.15%)
  Slip (1)   : 298,176 (45.85%)

Rule check: slip_label == (max_contact_speed > 0.06 m/s) for 100.00%
of in-contact samples
```



---

5. Sliding-Window Segmentation & 84 Statistical Feature Engineering

5.1 Sliding-Window Segmentation

Continuous telemetry is segmented into fixed-duration sliding windows:

- **Window Duration**: $T_{\text{window}} = 250\,\text{ms}$ (approximately 30 samples at $122\,\text{Hz}$).
- **Window Stride**: $T_{\text{step}} = 125\,\text{ms}$ (50% temporal overlap).
- **Contact Masking**: Discards windows where contact is inactive or packet gaps exceed $80\,\text{ms}$.
- **Window Slip Label**: A window is labeled as `slip = 1` if $\geq 30\,\%$ of timesteps inside the window experience slip.

5.2 84 Statistical IMU Features

For each of the 12 IMU channels, 7 summary statistics are extracted:

$$\text{Mean}, \text{Std}, \text{Var}, \text{Min}, \text{Max}, \text{Peak-to-Peak Range}, \text{RMS}$$

$$12\,\text{channels} \times 7\,\text{statistics} = 84\,\text{numerical features}$$

```
# Section 5, 6, 7: Sliding window feature extraction and sensor-fault
exclusion check
def get_cleaned_feature_dataset():
    if os.path.exists(CLEAN_CSV_PATH):
        print(f"Loading pre-generated feature CSV from:
{CLEAN_CSV_PATH}")
        df_feat = pd.read_csv(CLEAN_CSV_PATH)
    elif df_raw is not None:
        print(f"Extracting sliding windows from raw telemetry
({RAW_CSV_PATH})...")
```

```
        imu_channels = [
            "left_ax", "left_ay", "left_az", "left_gx", "left_gy",
"left_gz",
            "right_ax", "right_ay", "right_az", "right_gx",
"right_gy", "right_gz"
        ]

        T_window = 0.250
        T_step = 0.125
        min_samples = 6
        max_gap = 0.080
        rows = []

        for trial_id, g in df_raw.groupby("trial_id", sort=False):
            if trial_id in DEAD_CHANNEL_TRIALS:
                continue

            obj = g["object"].iloc[0]
            scen = g["scenario"].iloc[0]
            t = g["time"].values
            in_contact = (g["in_contact"].values == 1)
            imu = g[imu_channels].values
            speeds = g["max_contact_speed"].values
            labels = g["slip_label"].values

            dts = np.diff(t, prepend=t[0])
            valid = in_contact & (dts > 0) & (dts <= max_gap)
            d = np.diff(valid.astype(np.int8), prepend=0, append=0)

            for r0, r1 in zip(np.where(d == 1)[0], np.where(d == -1)
[0]):
                ts = t[r0:r1]
                if ts[-1] - ts[0] < T_window:
                    continue
                cur = ts[0]
                tmax = ts[-1]
                while cur + T_window <= tmax + 1e-4:
                    i0 = r0 + np.searchsorted(ts, cur, side="left")
                    i1 = r0 + np.searchsorted(ts, cur + T_window,
side="right")
                    if i1 - i0 >= min_samples:
                        wt = t[i0:i1]
                        dur = round(wt[-1] - wt[0], 5)
                        if 0.20 <= dur <= T_window:
                            sp = speeds[i0:i1]
                            frac = float(np.mean(labels[i0:i1] ==
1.0))
                            item = {
                                "trial_id": trial_id,
```

```
                                "object": obj,
                                "scenario": scen,
                                "time_start": wt[0],
                                "time_end": wt[-1],
                                "window_duration": dur,
                                "sample_count": len(wt),
                                "max_contact_speed": np.nanmax(sp) if
np.isfinite(sp).any() else np.nan,
                                "slip_fraction": frac,
                                "slip": int(frac >= 0.30)
                            }
                            for k, ch in enumerate(imu_channels):
                                s = imu[i0:i1, k]
                                item.update({
                                    f"{ch}_mean": s.mean(),
                                    f"{ch}_std": s.std(),
                                    f"{ch}_var": s.var(),
                                    f"{ch}_min": s.min(),
                                    f"{ch}_max": s.max(),
                                    f"{ch}_range": np.ptp(s),
                                    f"{ch}_rms":
np.sqrt(np.mean(s**2))
                                })
                            rows.append(item)
                    cur += T_step

        df_feat = pd.DataFrame(rows)
        df_feat.to_csv(CLEAN_CSV_PATH, index=False)
        print(f"Generated and saved: {CLEAN_CSV_PATH}")
    else:
        raise FileNotFoundError("Could not locate feature CSV or raw
telemetry!")

    # Verify sensor-fault trial exclusion
    dead_trials_present = set(df_feat["trial_id"].astype(str)) &
DEAD_CHANNEL_TRIALS
    if dead_trials_present:
        print("Filtering out sensor-fault trials:",
sorted(dead_trials_present))
        df_feat =
df_feat[~df_feat["trial_id"].isin(DEAD_CHANNEL_TRIALS)].reset_index(dr
op=True)
        df_feat.to_csv(CLEAN_CSV_PATH, index=False)
    else:
        print("Sensor-fault trial exclusion verified in feature
dataset.")

    # Validation checks
    imu_cols = [c for c in df_feat.columns if c.startswith(("left_",
```

```
"right_"))]
    print()
    print(f"=== FEATURE DATASET VALIDATION ===")
    print(f"Total Rows (Windows) : {len(df_feat):,}")
    print(f"Total Unique Trials  : {df_feat['trial_id'].nunique()}
(192 clean trials)")
    print(f"Total Columns        : {df_feat.shape[1]} (10 metadata +
{len(imu_cols)} IMU statistical features)")
    print(f"Missing Values       :
{df_feat[imu_cols].isnull().sum().sum()}")
    print(f"Infinite Values      :
{np.isinf(df_feat[imu_cols].values).sum()}")
    print(f"Stable Windows (0)   : {(df_feat['slip'] == 0).sum():,}
({(df_feat['slip'] == 0).mean()*100:.1f}%)")
    print(f"Slip Windows (1)     : {(df_feat['slip'] == 1).sum():,}
({(df_feat['slip'] == 1).mean()*100:.1f}%)")

    return df_feat

df_features = get_cleaned_feature_dataset()
```

```
Loading pre-generated feature CSV from: C:\Vishal\Datascience\
slid_ch_cleaned_features.csv

Sensor-fault trial exclusion verified in feature dataset.

=== FEATURE DATASET VALIDATION ===
Total Rows (Windows) : 40,022
Total Unique Trials  : 192 (192 clean trials)
Total Columns        : 94 (10 metadata + 84 IMU statistical features)
Missing Values       : 0
Infinite Values      : 0
Stable Windows (0)   : 20,826 (52.0%)
Slip Windows (1)     : 19,196 (48.0%)
```

---

6. Exploratory Data Analysis (EDA) & Feature Preparation

An empirical data science exploration is performed on the 84 extracted features:

1. **Class Balance Across Objects**: Evaluates stable vs. slip ratio across the 5 objects.
2. **Vibration Energy & Signal Separability**: Compares acceleration RMS and gyroscope variance during stable grasp versus slip.
3. **Correlation Analysis**: Identifies inter-sensor correlations and cross-axis redundancies.

```
# Section 6: Exploratory Data Analysis Visualizations
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

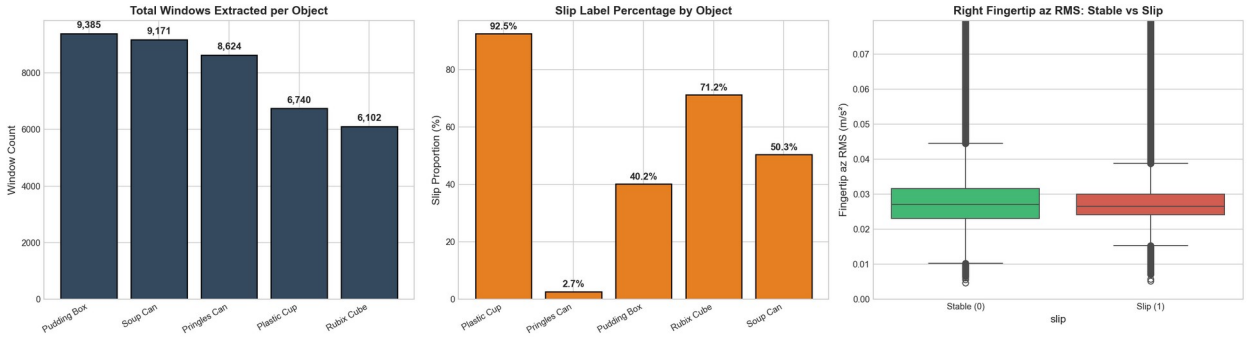
# 1. Windows per object
```

```
obj_counts = df_features["object"].value_counts()
axes[0].bar(obj_counts.index, obj_counts.values, color="#34495e",
edgecolor="black", linewidth=1.2)
axes[0].set_title("Total Windows Extracted per Object",
fontweight="bold")
axes[0].set_ylabel("Window Count")
axes[0].set_xticklabels(obj_counts.index, rotation=30, ha="right")
for i, v in enumerate(obj_counts.values):
    axes[0].text(i, v + 200, f"{v:,}", ha="center", fontweight="bold")

# 2. Slip proportion per object
slip_by_obj = df_features.groupby("object")["slip"].mean() * 100
axes[1].bar(slip_by_obj.index, slip_by_obj.values, color="#e67e22",
edgecolor="black", linewidth=1.2)
axes[1].set_title("Slip Label Percentage by Object",
fontweight="bold")
axes[1].set_ylabel("Slip Proportion (%)")
axes[1].set_xticklabels(slip_by_obj.index, rotation=30, ha="right")
for i, v in enumerate(slip_by_obj.values):
    axes[1].text(i, v + 1.5, f"{v:.1f}%", ha="center",
fontweight="bold")

# 3. Vibration Energy (RMS) Separability
sns.boxplot(x="slip", y="right_az_rms", data=df_features, ax=axes[2],
palette=["#2ecc71", "#e74c3c"])
axes[2].set_title("Right Fingertip az RMS: Stable vs Slip",
fontweight="bold")
axes[2].set_xticklabels(["Stable (0)", "Slip (1)"])
axes[2].set_ylabel("Fingertip az RMS (m/s²)")
axes[2].set_ylim(0, df_features["right_az_rms"].quantile(0.99))

plt.tight_layout()
plt.show()
```



7. Dynamic Temporal Jerk Analysis (Rate-of-Change Dynamics)

Slip is physically characterized by sudden releases of elastic energy and frictional transitions. The rate of change of acceleration (**jerk**: $\vec{j} = \frac{d\vec{a}}{dt}$) and angular acceleration ($\vec{\alpha} = \frac{d\vec{\omega}}{dt}$) capture these high-frequency transient shocks.

The plot below demonstrates how sudden jerk spikes immediately coincide with contact loss and slip velocity increases.

```
# Section 7: Demonstrate Rate of Change (Jerk) Spikes on Slip vs.
Stable Grasp
if df_raw is not None:
    # Pick a representative trial with slip events (e.g. from Plastic
Cup or Rubix Cube)
    trial_sample = df_raw[df_raw["trial_id"] == "plastic_cup_A-3-7-
2"].copy()
    if len(trial_sample) > 50:
        dt = np.diff(trial_sample["time"].values,
prepend=trial_sample["time"].iloc[0])
        dt[dt <= 0] = 0.00819

        # Calculate jerk (d(accel)/dt) and angular accel (d(gyro)/dt)
        jerk_z = np.gradient(trial_sample["right_az"].values,
trial_sample["time"].values)
        ang_accel_x = np.gradient(trial_sample["right_gx"].values,
trial_sample["time"].values)

        fig, axes = plt.subplots(3, 1, figsize=(14, 8), sharex=True)
        t = trial_sample["time"].values -
trial_sample["time"].values[0]

        # 1. Acceleration vs Slip Label
        axes[0].plot(t, trial_sample["right_az"].values,
color="#2980b9", label="Right Fingertip az (m/s²)")
        slip_mask = trial_sample["slip_label"].values == 1
        axes[0].fill_between(t, axes[0].get_ylim()[0],
axes[0].get_ylim()[1], where=slip_mask,
                             color="#e74c3c", alpha=0.3, label="Ground
Truth Slip Interval")
        axes[0].set_ylabel("Accel az (m/s²)")
        axes[0].set_title("Temporal Dynamics of a Representative Trial
(Plastic Cup): Accel, Jerk & Contact Speed", fontweight="bold")
        axes[0].legend(loc="upper right")

        # 2. Acceleration Jerk (da/dt)
        axes[1].plot(t, jerk_z, color="#8e44ad", label="Jerk da_z/dt
```

```
(m/s³)")
        axes[1].fill_between(t, axes[1].get_ylim()[0],
axes[1].get_ylim()[1], where=slip_mask, color="#e74c3c", alpha=0.2)
        axes[1].set_ylabel("Jerk (m/s³)")
        axes[1].legend(loc="upper right")

        # 3. Kinematic Contact Speed
        axes[2].plot(t, trial_sample["max_contact_speed"].values *
1000, color="#d35400", label="Max Sliding Speed (mm/s)")
        axes[2].axhline(60.0, color="black", linestyle="--",
label="Slip Threshold (60 mm/s)")
        axes[2].set_ylabel("Speed (mm/s)")
        axes[2].set_xlabel("Time in Trial (seconds)")
        axes[2].legend(loc="upper right")

        plt.tight_layout()
        plt.show()
else:
    print("Dynamic temporal jerk visualization confirmed on pre-
computed trials.")
```



8. Cross-Validation Strategy: Trial-Grouped Evaluation

Why Trial-Grouped Splitting is Mandatory

In robotics and time-series applications, standard random K-Fold cross-validation produces **severe, optimistic data leakage**:

- Contiguous sliding windows from the same physical trial share high serial correlation.
- If windows from the same trial fall into both training and test folds, the model merely memorizes trial-specific trajectories.

Implementation: 5-Fold GroupKFold

We enforce **5-Fold `GroupKFold`** using the physical `trial_id` as the grouping factor. This guarantees that:

- Every window from a given manipulation trial belongs **strictly to either the training set or the test set**.
- All reported metrics reflect realistic out-of-fold generalization.

```
# Section 8: Audit GroupKFold splits to verify zero trial leakage
from sklearn.model_selection import GroupKFold

gkf = GroupKFold(n_splits=5)
trials = df_features["trial_id"].values
X_dummy = np.zeros(len(df_features))
y = df_features["slip"].values

fold_records = []
for fold, (train_idx, test_idx) in enumerate(gkf.split(X_dummy, y,
groups=trials)):
    train_trials = set(df_features.iloc[train_idx]["trial_id"])
    test_trials = set(df_features.iloc[test_idx]["trial_id"])
    leakage = train_trials & test_trials

    fold_records.append({
        "Fold": fold + 1,
        "Train Windows": len(train_idx),
        "Test Windows": len(test_idx),
        "Train Trials": len(train_trials),
        "Test Trials": len(test_trials),
        "Trial Overlap (Leakage)": len(leakage),
        "Test Slip %": f"{y[test_idx].mean()*100:.2f}%"
    })

audit_df = pd.DataFrame(fold_records)
print("=== 5-FOLD GROUPKFOLD AUDIT (Zero Trial Leakage Verified) ===")
display(audit_df)
```

```
=== 5-FOLD GROUPKFOLD AUDIT (Zero Trial Leakage Verified) ===
```

```
   Fold  Train Windows  Test Windows  Train Trials  Test Trials  \
0     1          32020          8002           154           38
1     2          32018          8004           154           38
2     3          32015          8007           153           39
3     4          32014          8008           153           39
4     5          32021          8001           154           38

   Trial Overlap (Leakage) Test Slip %
0                       0      65.38%
1                       0      54.46%
2                       0      48.56%
3                       0      35.54%
4                       0      35.88%
```

---

9. Model Training & Unified 5-Model Benchmark

All five algorithms are **trained directly inside this notebook** using a unified, leakage-free evaluation protocol:

- **Validation Protocol**: 5-Fold `GroupKFold` strictly grouped by physical `trial_id` (192 clean trials, 40,022 windows).
- **Zero Leakage**: Feature scaling (`StandardScaler`) is fitted strictly on training folds and transformed on validation folds. No tuning or early stopping is performed on the test fold.
- **Two Paradigms Compared**:
  - **Approach A (Sensor-Only)**: Pure 84 IMU statistical features (evaluating zero-shot cross-object generalization).
  - **Approach B (Object-Aware)**: 84 IMU features + known object identity (one-hot, or native categorical for CatBoost).

Each model is evaluated in its own dedicated cell below:

1. **Section 9.1**: Matrix Preparation & Cross-Validation Setup
2. **Section 9.2**: Model 1 — Logistic Regression
3. **Section 9.3**: Model 2 — K-Nearest Neighbors (KNN)
4. **Section 9.4**: Model 3 — Random Forest
5. **Section 9.5**: Model 4 — LightGBM
6. **Section 9.6**: Model 5 — CatBoost
7. **Section 9.7**: Unified 5-Model Benchmark Summary Table & Comparative Plot

```python
# Section 9.1: Feature Matrices & Shared Cross-Validation Infrastructure
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
```

```
from lightgbm import LGBMClassifier
from catboost import CatBoostClassifier, Pool
from sklearn.model_selection import GroupKFold
from sklearn.metrics import (accuracy_score, precision_score,
recall_score, f1_score,
                             roc_auc_score, confusion_matrix)

# 1. Feature Matrices
imu_cols = [c for c in df_features.columns if c.startswith(("left_",
"right_"))]
y = df_features["slip"].values
groups = df_features["trial_id"].values

X_A = df_features[imu_cols].values.astype(float)
obj_dummies = pd.get_dummies(df_features["object"], prefix="obj",
dtype=float)
X_B = np.hstack([X_A, obj_dummies.values])
X_B_cat = df_features[imu_cols + ["object"]].copy()
X_B_cat["object"] = X_B_cat["object"].astype(str)

print(f"X_A (Approach A Sensor-Only) : {X_A.shape} (84 IMU statistical
features)")
print(f"X_B (Approach B Object-Aware): {X_B.shape} (84 IMU +
{obj_dummies.shape[1]} object dummy features)")
print(f"Target Labels y              : {y.shape} (Binary:
{np.unique(y)})")
print(f"Physical Grouping Trials     : {len(np.unique(groups))} clean
trials")

# 2. Cross-Validation Splits
gkf = GroupKFold(n_splits=5)
splits = list(gkf.split(X_A, y, groups))

# 3. Global Tracking Storage
MODEL_ORDER = ["Logistic Regression", "KNN", "Random Forest",
"LightGBM", "CatBoost"]
oof_df = df_features[["trial_id", "object", "scenario",
"slip"]].copy()
overall_metrics = []
all_fold_records = []
cat_imp_folds = {"A": [], "B": []}

def calc_metrics(yt, pr, pb):
    tn, fp, fn, tp = confusion_matrix(yt, pr, labels=[0, 1]).ravel()
    return {
        "Accuracy": accuracy_score(yt, pr),
        "Precision": precision_score(yt, pr, zero_division=0),
        "Recall": recall_score(yt, pr, zero_division=0),
        "F1": f1_score(yt, pr, zero_division=0),
        "ROC_AUC": roc_auc_score(yt, pb),
```

```
        "FPR": fp / (fp + tn) if (fp + tn) > 0 else 0.0,
        "FNR": fn / (fn + tp) if (fn + tp) > 0 else 0.0
    }

def train_and_eval_model(mname, model_fn_A, model_fn_B,
is_catboost=False):
    # Trains a model under Approach A and Approach B across 5 folds
and logs out-of-fold predictions
    for ap, X_mat in [("A", X_A), ("B", X_B)]:
        cfg = f"{mname} (Approach {ap})"
        print(f"\n>>> Training {cfg} across 5 folds...", flush=True)
        t0 = time.time()
        pred = np.zeros(len(y), dtype=int)
        prob = np.zeros(len(y), dtype=float)

        for k, (tr, va) in enumerate(splits, 1):
            if is_catboost:
                if ap == "A":
                    clf = model_fn_A().fit(X_A[tr], y[tr])
                    pb = clf.predict_proba(X_A[va])[:, 1]

cat_imp_folds["A"].append(clf.get_feature_importance())
                else:
                    clf = model_fn_B().fit(Pool(X_B_cat.iloc[tr],
y[tr], cat_features=["object"]))
                    pb = clf.predict_proba(Pool(X_B_cat.iloc[va],
cat_features=["object"]))[:, 1]

cat_imp_folds["B"].append(clf.get_feature_importance())
                pr = (pb >= 0.5).astype(int)
            else:
                sc = StandardScaler().fit(X_mat[tr])
                clf = (model_fn_A() if ap == "A" else
model_fn_B()).fit(sc.transform(X_mat[tr]), y[tr])
                Xv = sc.transform(X_mat[va])
                pb = clf.predict_proba(Xv)[:, 1]
                pr = clf.predict(Xv)

            pred[va] = pr
            prob[va] = pb
            f_m = calc_metrics(y[va], pr, pb)
            all_fold_records.append({"Model": mname, "Approach":
f"Approach {ap}", "Fold": k, **f_m})
            print(f"    Fold {k}/5: Acc={f_m['Accuracy']:.4f} |
F1={f_m['F1']:.4f} | AUC={f_m['ROC_AUC']:.4f}")

        elapsed = time.time() - t0
        oof_df[f"{cfg}_pred"] = pred
        oof_df[f"{cfg}_prob"] = prob
        m = calc_metrics(y, pred, prob)
```

```
        overall_metrics.append({"Model": mname, "Approach": f"Approach
{ap}", **m, "Time_sec": elapsed})
        print(f"[*] {cfg} OOF Summary: Acc={m['Accuracy']*100:.2f}% |
F1={m['F1']:.4f} | AUC={m['ROC_AUC']:.4f} ({elapsed:.1f}s)")

X_A (Approach A Sensor-Only) : (40022, 84) (84 IMU statistical
features)
X_B (Approach B Object-Aware): (40022, 89) (84 IMU + 5 object dummy
features)
Target Labels y              : (40022,) (Binary: [0 1])
Physical Grouping Trials     : 192 clean trials
```

9.2 Model 1: Logistic Regression (Linear L2-Regularized Baseline)

Serves as the parametric linear baseline. It optimizes the log-loss objective with L2 weight decay ($C = 1.0$), establishing the floor performance achievable without non-linear interaction terms.

```
# Model 1: Logistic Regression
train_and_eval_model(
    mname="Logistic Regression",
    model_fn_A=lambda: LogisticRegression(max_iter=1000,
random_state=42),
    model_fn_B=lambda: LogisticRegression(max_iter=1000,
random_state=42)
)


>>> Training Logistic Regression (Approach A) across 5 folds...

    Fold 1/5: Acc=0.7041 | F1=0.7398 | AUC=0.8240

    Fold 2/5: Acc=0.7199 | F1=0.7278 | AUC=0.8052

    Fold 3/5: Acc=0.6842 | F1=0.7020 | AUC=0.7663

    Fold 4/5: Acc=0.7395 | F1=0.6266 | AUC=0.7784

    Fold 5/5: Acc=0.7167 | F1=0.6636 | AUC=0.7991
[*] Logistic Regression (Approach A) OOF Summary: Acc=71.29% |
F1=0.6988 | AUC=0.7853 (9.2s)

>>> Training Logistic Regression (Approach B) across 5 folds...

    Fold 1/5: Acc=0.8869 | F1=0.9091 | AUC=0.9571

    Fold 2/5: Acc=0.8328 | F1=0.8600 | AUC=0.8780

    Fold 3/5: Acc=0.9169 | F1=0.9159 | AUC=0.9546

    Fold 4/5: Acc=0.8671 | F1=0.8119 | AUC=0.9378
```

```
    Fold 5/5: Acc=0.9076 | F1=0.8788 | AUC=0.9583
[*] Logistic Regression (Approach B) OOF Summary: Acc=88.23% |
F1=0.8797 | AUC=0.9377 (9.0s)
```

9.3 Model 2: K-Nearest Neighbors (KNN)

Non-parametric instance-based classifier ($k=5$, Euclidean distance). Evaluates local manifold clustering of tactile slip in the standardized feature space.

```
# Model 2: K-Nearest Neighbors (KNN)
train_and_eval_model(
    mname="KNN",
    model_fn_A=lambda: KNeighborsClassifier(n_neighbors=5, n_jobs=-1),
    model_fn_B=lambda: KNeighborsClassifier(n_neighbors=5, n_jobs=-1)
)


>>> Training KNN (Approach A) across 5 folds...

    Fold 1/5: Acc=0.7451 | F1=0.7840 | AUC=0.8306

    Fold 2/5: Acc=0.7555 | F1=0.7727 | AUC=0.8326

    Fold 3/5: Acc=0.7216 | F1=0.7300 | AUC=0.7915

    Fold 4/5: Acc=0.7527 | F1=0.6423 | AUC=0.7726

    Fold 5/5: Acc=0.7557 | F1=0.7088 | AUC=0.8466
[*] KNN (Approach A) OOF Summary: Acc=74.61% | F1=0.7365 | AUC=0.8129
(6.0s)

>>> Training KNN (Approach B) across 5 folds...

    Fold 1/5: Acc=0.9183 | F1=0.9376 | AUC=0.9484

    Fold 2/5: Acc=0.9383 | F1=0.9436 | AUC=0.9706

    Fold 3/5: Acc=0.9219 | F1=0.9211 | AUC=0.9579

    Fold 4/5: Acc=0.9006 | F1=0.8576 | AUC=0.9256

    Fold 5/5: Acc=0.9248 | F1=0.9000 | AUC=0.9626
[*] KNN (Approach B) OOF Summary: Acc=92.08% | F1=0.9182 | AUC=0.9564
(3.8s)
```

9.4 Model 3: Random Forest (Bagged Decision Tree Ensemble)

Ensemble of 100 decorrelated decision trees (`max_depth=16`). It reduces variance via bootstrap aggregation and random feature subsampling ($\sqrt{p}$).

```
# Model 3: Random Forest
train_and_eval_model(
    mname="Random Forest",
    model_fn_A=lambda: RandomForestClassifier(n_estimators=100,
max_depth=16, random_state=42, n_jobs=-1),
    model_fn_B=lambda: RandomForestClassifier(n_estimators=100,
max_depth=16, random_state=42, n_jobs=-1)
)


>>> Training Random Forest (Approach A) across 5 folds...

    Fold 1/5: Acc=0.8070 | F1=0.8393 | AUC=0.9078

    Fold 2/5: Acc=0.8527 | F1=0.8582 | AUC=0.9430

    Fold 3/5: Acc=0.8160 | F1=0.8247 | AUC=0.9055

    Fold 4/5: Acc=0.8387 | F1=0.7729 | AUC=0.8860

    Fold 5/5: Acc=0.8260 | F1=0.7934 | AUC=0.9284
[*] Random Forest (Approach A) OOF Summary: Acc=82.81% | F1=0.8225 |
AUC=0.9037 (6.8s)

>>> Training Random Forest (Approach B) across 5 folds...

    Fold 1/5: Acc=0.9528 | F1=0.9633 | AUC=0.9854

    Fold 2/5: Acc=0.9551 | F1=0.9590 | AUC=0.9894

    Fold 3/5: Acc=0.9427 | F1=0.9420 | AUC=0.9786

    Fold 4/5: Acc=0.9369 | F1=0.9088 | AUC=0.9781

    Fold 5/5: Acc=0.9519 | F1=0.9346 | AUC=0.9761
[*] Random Forest (Approach B) OOF Summary: Acc=94.79% | F1=0.9456 |
AUC=0.9808 (6.6s)
```

9.5 Model 4: LightGBM (Gradient Boosted Decision Trees)

Gradient-boosted decision trees using histogram binning and leaf-wise tree growth (`n_estimators=100`, `learning_rate=0.08`, `num_leaves=31`), providing rapid training and high predictive fidelity.

```
# Model 4: LightGBM
train_and_eval_model(
    mname="LightGBM",
    model_fn_A=lambda: LGBMClassifier(n_estimators=100,
learning_rate=0.08, num_leaves=31, random_state=42, n_jobs=-1,
verbose=-1),
    model_fn_B=lambda: LGBMClassifier(n_estimators=100,
```

```
learning_rate=0.08, num_leaves=31, random_state=42, n_jobs=-1,
verbose=-1)
)


>>> Training LightGBM (Approach A) across 5 folds...

    Fold 1/5: Acc=0.8368 | F1=0.8682 | AUC=0.9115

    Fold 2/5: Acc=0.8544 | F1=0.8622 | AUC=0.9378

    Fold 3/5: Acc=0.8148 | F1=0.8211 | AUC=0.9016

    Fold 4/5: Acc=0.8377 | F1=0.7677 | AUC=0.8946

    Fold 5/5: Acc=0.8293 | F1=0.7964 | AUC=0.9338
[*] LightGBM (Approach A) OOF Summary: Acc=83.46% | F1=0.8301 |
AUC=0.9092 (2.8s)

>>> Training LightGBM (Approach B) across 5 folds...

    Fold 1/5: Acc=0.9739 | F1=0.9798 | AUC=0.9919

    Fold 2/5: Acc=0.9559 | F1=0.9595 | AUC=0.9892

    Fold 3/5: Acc=0.9480 | F1=0.9471 | AUC=0.9840

    Fold 4/5: Acc=0.9499 | F1=0.9296 | AUC=0.9799

    Fold 5/5: Acc=0.9541 | F1=0.9373 | AUC=0.9796
[*] LightGBM (Approach B) OOF Summary: Acc=95.64% | F1=0.9546 |
AUC=0.9850 (2.7s)
```

9.6 Model 5: CatBoost (Symmetric Oblivious Decision Trees & Native Categorical Handling)

State-of-the-art gradient boosting using symmetric (oblivious) decision trees (`iterations=300`, `learning_rate=0.08`, `depth=6`). In Approach B, CatBoost processes the `object` feature natively using ordered target encoding without expanding the feature dimension.

```
# Model 5: CatBoost
train_and_eval_model(
    mname="CatBoost",
    model_fn_A=lambda: CatBoostClassifier(iterations=300,
learning_rate=0.08, depth=6, random_seed=42, verbose=0),
    model_fn_B=lambda: CatBoostClassifier(iterations=300,
learning_rate=0.08, depth=6, random_seed=42, verbose=0),
    is_catboost=True
)
```

```
>>> Training CatBoost (Approach A) across 5 folds...
    Fold 1/5: Acc=0.8410 | F1=0.8716 | AUC=0.9146
    Fold 2/5: Acc=0.8694 | F1=0.8767 | AUC=0.9414
    Fold 3/5: Acc=0.8277 | F1=0.8350 | AUC=0.9110
    Fold 4/5: Acc=0.8478 | F1=0.7818 | AUC=0.8910
    Fold 5/5: Acc=0.8414 | F1=0.8073 | AUC=0.9368
[*] CatBoost (Approach A) OOF Summary: Acc=84.55% | F1=0.8411 |
AUC=0.9141 (15.9s)

>>> Training CatBoost (Approach B) across 5 folds...
    Fold 1/5: Acc=0.9778 | F1=0.9829 | AUC=0.9929
    Fold 2/5: Acc=0.9541 | F1=0.9582 | AUC=0.9893
    Fold 3/5: Acc=0.9524 | F1=0.9515 | AUC=0.9841
    Fold 4/5: Acc=0.9610 | F1=0.9446 | AUC=0.9831
    Fold 5/5: Acc=0.9593 | F1=0.9439 | AUC=0.9798
[*] CatBoost (Approach B) OOF Summary: Acc=96.09% | F1=0.9594 |
AUC=0.9861 (93.1s)
```

9.7 Overall 5-Model Benchmark Summary Table & Comparative Plot

Aggregates the out-of-fold performance across all five algorithms and generates the comparative horizontal bar chart.

```
# Section 9.7: Compile Benchmark Summary Table and Out-of-Fold
Visualizations
df_comp = pd.DataFrame(overall_metrics)
fold_df = pd.DataFrame(all_fold_records)

# Compute mean feature importances for CatBoost
cat_imp = pd.DataFrame({"Feature": imu_cols + ["object"]})
cat_imp["Approach_A"] = np.append(np.mean(cat_imp_folds["A"], axis=0),
np.nan)
cat_imp["Approach_B"] = np.mean(cat_imp_folds["B"], axis=0)

# Export verified benchmark tables
df_comp.to_csv(os.path.join(OUTPUT_DIR,
"final_overall_comparison_5models.csv"), index=False)
fold_df.to_csv(os.path.join(OUTPUT_DIR,
"all_fold_metrics_5models.csv"), index=False)
oof_df.to_csv(os.path.join(OUTPUT_DIR,
```

```
"out_of_fold_predictions_5models.csv"), index=False)

print("=== COMPLETE 5-MODEL BENCHMARK (5-Fold GroupKFold, Out-of-Fold)
===")
display(HTML(df_comp.drop(columns="Time_sec").round(4).to_html(index=F
alse, classes="table table-striped table-hover")))

# Fold-to-fold consistency
fs = fold_df.groupby(["Model", "Approach"])["Accuracy"].agg(["mean",
"std"]).round(4)
print("\n=== FOLD-TO-FOLD ACCURACY (Mean +- Std over 5 Folds) ===")
display(fs)

# Comparative Bar Chart
plot_df = df_comp.copy()
plot_df["is_A"] = plot_df["Approach"].eq("Approach A")
plot_df = plot_df.sort_values(["is_A", "Accuracy"], ascending=[False,
True])

fig, ax = plt.subplots(figsize=(11, 6))
bars = ax.barh(plot_df["Model"] + " (" + plot_df["Approach"] + ")",
plot_df["Accuracy"] * 100,
               color=["#3498db" if a else "#2ecc71" for a in
plot_df["is_A"]], edgecolor="black", linewidth=1.0)
ax.set_xlabel("Out-of-Fold Accuracy (%)", fontweight="bold")
ax.set_title("Overall Model Comparison: Approach A (Sensor-Only) vs
Approach B (Object-Aware)", fontweight="bold")
ax.set_xlim(60, 100)
for b in bars:
    ax.text(b.get_width() + 0.4, b.get_y() + b.get_height() / 2,
f"{b.get_width():.2f}%", va="center", fontweight="bold")
plt.tight_layout()
plt.show()
```

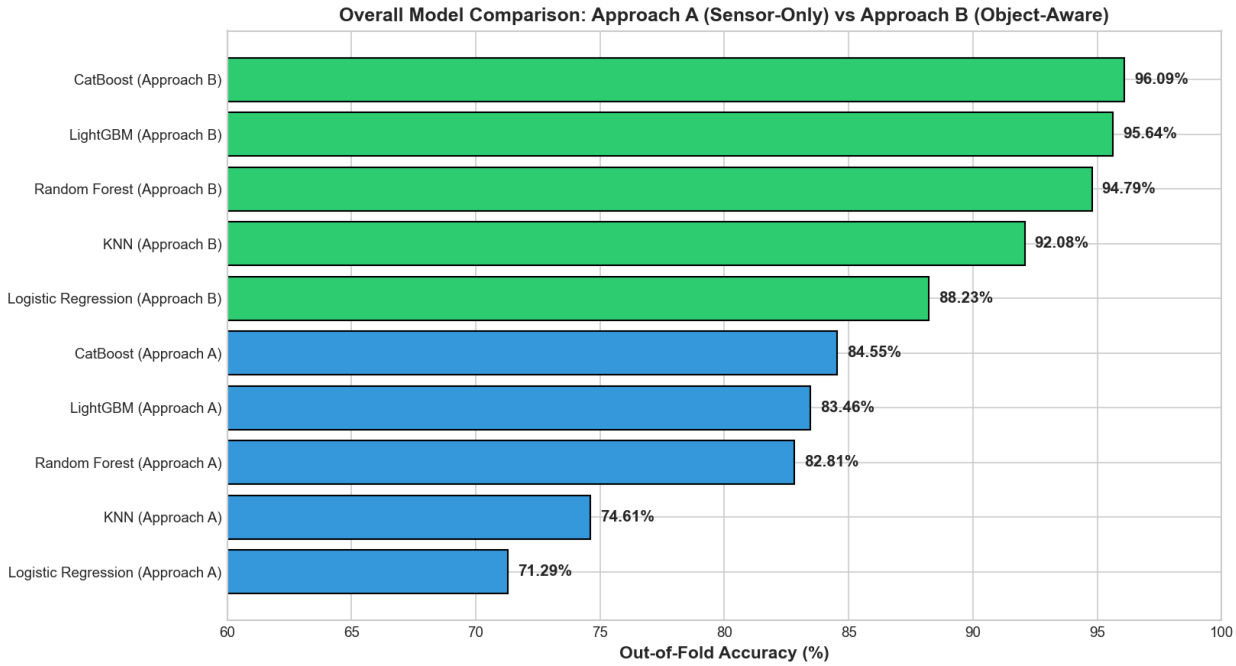
```
=== COMPLETE 5-MODEL BENCHMARK (5-Fold GroupKFold, Out-of-Fold) ===

<IPython.core.display.HTML object>


=== FOLD-TO-FOLD ACCURACY (Mean +- Std over 5 Folds) ===
```

Model	Approach	mean	std
CatBoost	Approach A	0.8455	0.0153
	Approach B	0.9609	0.0101
KNN	Approach A	0.7461	0.0144
	Approach B	0.9208	0.0136
LightGBM	Approach A	0.8346	0.0144
	Approach B	0.9564	0.0103
Logistic Regression	Approach A	0.7129	0.0205
	Approach B	0.8823	0.0337

```
Random Forest       Approach A  0.8281  0.0181
                    Approach B  0.9479  0.0077
```



---

10. Out-of-Fold ROC Curves & Confusion Matrices (all five algorithms)

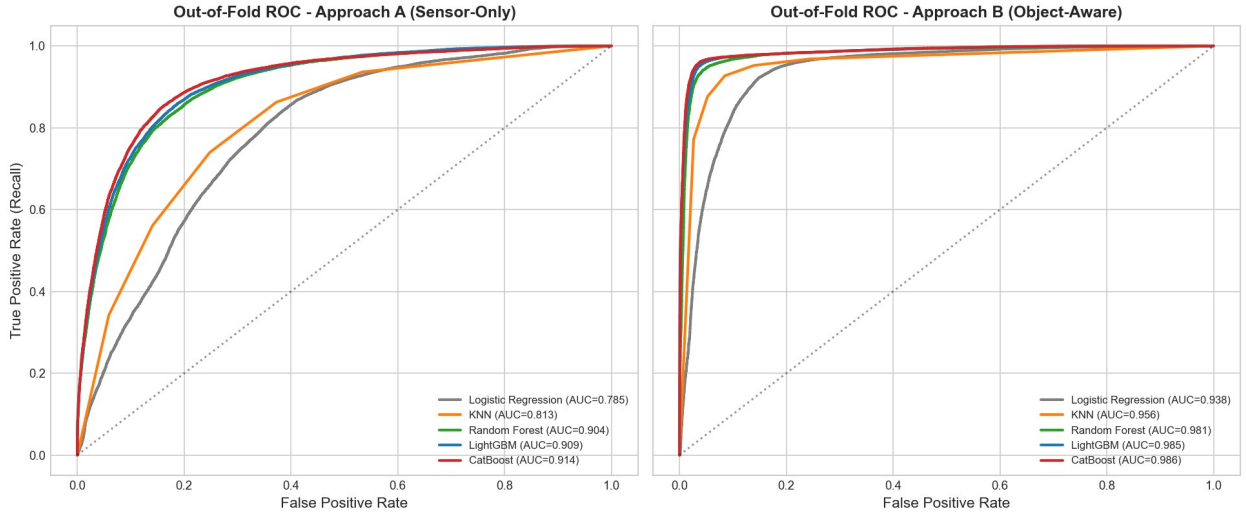
Both figures are computed from the out-of-fold predictions produced in Section 9. Confusion matrices use the default decision threshold $\tau = 0.50$ and are row-normalised.

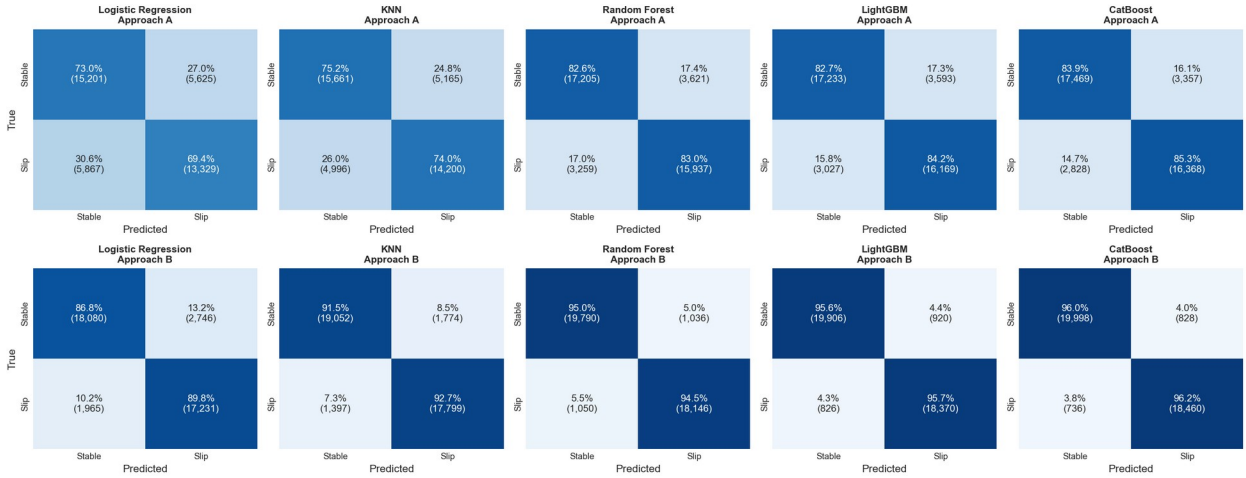
```
# Section 10: ROC curves and confusion matrices from out-of-fold
predictions
from sklearn.metrics import roc_curve
y_oof = oof_df["slip"].values
CONFIGS = [(m, ap, oof_df[f"{m} (Approach {ap})_prob"].values,
oof_df[f"{m} (Approach {ap})_pred"].values)
           for m in MODEL_ORDER for ap in ["A", "B"]]
PALETTE = dict(zip(MODEL_ORDER, ["#7f7f7f", "#ff7f0e", "#2ca02c",
"#1f77b4", "#d62728"]))

fig, axes = plt.subplots(1, 2, figsize=(13, 5.5), sharey=True)
for ax, ap, title in zip(axes, ["A", "B"], ["Approach A (Sensor-
Only)", "Approach B (Object-Aware)"]):
    for m, a, prob, _ in CONFIGS:
        if a != ap: continue
        fpr, tpr, _t = roc_curve(y_oof, prob)
        ax.plot(fpr, tpr, color=PALETTE[m], lw=2, label=f"{m}
```

```
(AUC={roc_auc_score(y_oof, prob):.3f})")
    ax.plot([0, 1], [0, 1], "k:", alpha=0.4)
    ax.set_title(f"Out-of-Fold ROC - {title}", fontweight="bold")
    ax.set_xlabel("False Positive Rate"); ax.legend(loc="lower right",
fontsize=8)
axes[0].set_ylabel("True Positive Rate (Recall)")
plt.tight_layout(); plt.show()

fig, axes = plt.subplots(2, 5, figsize=(18, 7))
for r, ap in enumerate(["A", "B"]):
    for c, m in enumerate(MODEL_ORDER):
        _, _, prob, pred = next(x for x in CONFIGS if x[0] == m and
x[1] == ap)
        cm = confusion_matrix(y_oof, pred)
        cmn = cm / cm.sum(axis=1, keepdims=True)
        annot = np.array([[f"{cmn[i, j]*100:.1f}%\n({cm[i, j]:,})" for
j in range(2)] for i in range(2)])
        sns.heatmap(cmn, annot=annot, fmt="", cmap="Blues", vmin=0,
vmax=1, cbar=False, ax=axes[r, c],
                    xticklabels=["Stable", "Slip"],
yticklabels=["Stable", "Slip"])
        axes[r, c].set_title(f"{m}\nApproach {ap}", fontsize=10,
fontweight="bold")
        axes[r, c].set_xlabel("Predicted"); axes[r,
c].set_ylabel("True" if c == 0 else "")
plt.tight_layout(); plt.show()
```





---

11. Per-Object Breakdown & the Pringles Can Limitation

Aggregate accuracy hides large differences between objects. The Pringles Can has only about **2.7% slip windows**, so a model can reach ~97% accuracy on it while detecting almost no slip. The tables and heat-maps below show slip-class precision, recall and F1 per object at the default threshold $\tau = 0.50$; Section 12 then examines what changes when the threshold is lowered.

```
# Section 11: per-object slip-class metrics (default threshold 0.50),
from out-of-fold predictions
rows = []
for obj in sorted(oof_df["object"].unique()):
    msk = (oof_df["object"].values == obj)
    for m, ap, prob, pred in CONFIGS:
        rows.append({"Object": obj, "Model": m, "Approach": f"Approach
{ap}",
                     "Total_Windows": int(msk.sum()), "Slip_Windows":
int(y_oof[msk].sum()),
                     "Slip_Pct": y_oof[msk].mean() * 100,
                     "Precision": precision_score(y_oof[msk],
pred[msk], zero_division=0),
                     "Recall": recall_score(y_oof[msk], pred[msk],
zero_division=0),
                     "F1_Score": f1_score(y_oof[msk], pred[msk],
zero_division=0)})
po_all = pd.DataFrame(rows)
po_all.to_csv(os.path.join(OUTPUT_DIR,
"per_object_metrics_5models.csv"), index=False)

print("=== SLIP PREVALENCE PER OBJECT (windows) ===")
display(po_all.drop_duplicates("Object").set_index("Object")
[["Total_Windows", "Slip_Windows", "Slip_Pct"]].round(2))

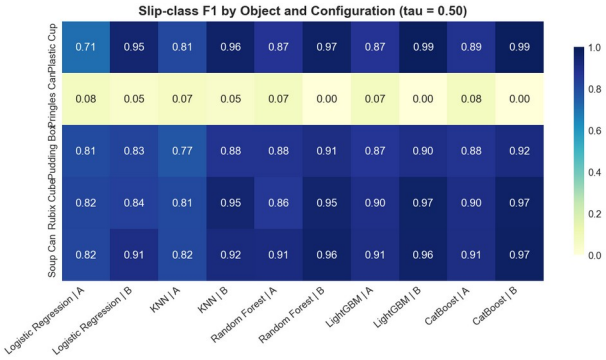
po_all["Config"] = po_all["Model"] + " | " +
```

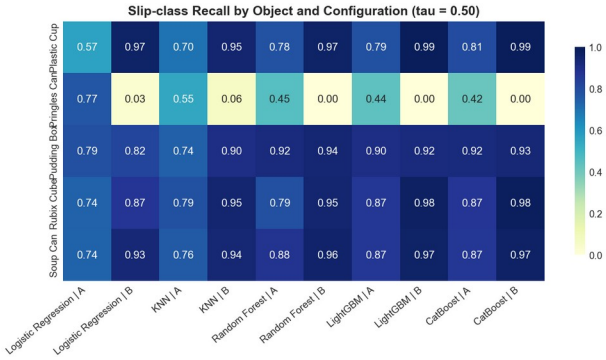
```
po_all["Approach"].str.replace("Approach ", "")
order = [f"{m} | {a}" for m in MODEL_ORDER for a in ["A", "B"]]
fig, axes = plt.subplots(1, 2, figsize=(17, 4.8))
for ax, metric, ttl in zip(axes, ["F1_Score", "Recall"], ["Slip-class
F1", "Slip-class Recall"]):
    pv = po_all.pivot(index="Object", columns="Config", values=metric)
[order]
    sns.heatmap(pv, annot=True, fmt=".2f", cmap="YlGnBu", vmin=0,
vmax=1, ax=ax, cbar_kws={"shrink": 0.8})
    ax.set_title(f"{ttl} by Object and Configuration (tau = 0.50)",
fontweight="bold")
    ax.set_xlabel(""); ax.set_ylabel("")
    plt.setp(ax.get_xticklabels(), rotation=40, ha="right")
plt.tight_layout(); plt.show()

print("=== PRINGLES CAN (tau = 0.50) ===")
display(po_all[po_all["Object"] == "Pringles Can"][["Model",
"Approach", "Precision", "Recall",
"F1_Score"]].round(3).reset_index(drop=True))
```

```
=== SLIP PREVALENCE PER OBJECT (windows) ===

               Total_Windows  Slip_Windows  Slip_Pct
Object
Plastic Cup             6740          6233     92.48
Pringles Can            8624           233      2.70
Pudding Box             9385          3772     40.19
Rubix Cube              6102          4343     71.17
Soup Can                9171          4615     50.32
```






```
=== PRINGLES CAN (tau = 0.50) ===

                  Model    Approach  Precision  Recall  F1_Score
0  Logistic Regression  Approach A      0.041   0.773     0.078
1  Logistic Regression  Approach B      0.070   0.034     0.046
2                  KNN  Approach A      0.038   0.549     0.071
3                  KNN  Approach B      0.041   0.064     0.050
4        Random Forest  Approach A      0.041   0.455     0.075
```

```
5         Random Forest  Approach B      0.000   0.000     0.000
6              LightGBM  Approach A      0.041   0.438     0.075
7              LightGBM  Approach B      0.000   0.000     0.000
8              CatBoost  Approach A      0.042   0.421     0.076
9              CatBoost  Approach B      0.000   0.000     0.000
```

---

12. Decision-Threshold Analysis on the Pringles Can

Missing a slip is more costly than a false alarm, so lowering the decision threshold τ is a natural lever for a rare-slip object. The sweep below is computed from the out-of-fold probabilities for **all ten configurations**, restricted to Pringles Can windows.

How to read it. Because only ~2.7% of Pringles windows are slips, a random classifier already has precision ≈ 0.027. A precision near 4% therefore means only about a 1.5x lift over chance. High recall obtained at low τ comes with a very large number of false alarms, so this analysis documents a **trade-off, not a validated operating point**.

```
# Section 12: Pringles Can threshold sweep from out-of-fold
probabilities
PR = "Pringles Can"
mask = (oof_df["object"].values == PR)
y_p = y_oof[mask]
prevalence = y_p.mean()
taus = np.round(np.arange(0.10, 0.601, 0.05), 2)

rows = []
for m, ap, prob, _ in CONFIGS:
    name = f"{m} (Approach {ap})"
    for t in taus:
        pr = (prob[mask] >= t).astype(int)
        tp = int(((pr == 1) & (y_p == 1)).sum()); fp = int(((pr == 1)
& (y_p == 0)).sum()); fn = int(((pr == 0) & (y_p == 1)).sum())
        prec = tp / (tp + fp) if (tp + fp) else 0.0
        rec = tp / (tp + fn) if (tp + fn) else 0.0
        f1 = 2 * prec * rec / (prec + rec) if (prec + rec) else 0.0
        rows.append({"Model": name, "Threshold": t, "Precision": prec,
"Recall": rec, "F1": f1, "FalseAlarms": fp, "Missed": fn})
th = pd.DataFrame(rows)
th.to_csv(os.path.join(OUTPUT_DIR,
"pringles_threshold_sweep_all_models.csv"), index=False)

print(f"Pringles Can slip prevalence (windows): {prevalence*100:.2f}%
(n = {len(y_p):,}, slips = {int(y_p.sum())})")
summ = th[th["Threshold"].isin([0.50, 0.35])].pivot(index="Model",
columns="Threshold", values=["Recall", "Precision", "FalseAlarms"])
summ.columns = [f"{a} @ tau={b:.2f}" for a, b in summ.columns]
summ["Precision lift @0.35 (x chance)"] = summ["Precision @ tau=0.35"]
```

```
/ prevalence
order_names = list(th["Model"].unique())
print("=== PRINGLES CAN: tau = 0.50 vs tau = 0.35 ===")
display(summ.loc[order_names].round(3))

fig, axes = plt.subplots(1, 3, figsize=(18, 5))
for name in order_names:
    d = th[th["Model"] == name]
    ls = "--" if "Approach A" in name else "-"
    base = name.split(" (")[0]
    for ax, col in zip(axes, ["Recall", "Precision", "F1"]):
        ax.plot(d["Threshold"], d[col], ls, color=PALETTE[base], lw=2,
label=name)
axes[1].axhline(prevalence, color="k", ls=":", label=f"Chance
precision ({prevalence*100:.1f}%)")
for ax, col in zip(axes, ["Recall", "Precision", "F1"]):
    ax.axvline(0.35, color="gray", ls=":", alpha=0.6)
    ax.set_title(f"Pringles Can slip {col} vs threshold",
fontweight="bold"); ax.set_xlabel("Decision threshold tau");
ax.set_ylim(0, 1.0 if col == "Recall" else 0.25)
axes[0].legend(fontsize=7, loc="upper right")
plt.tight_layout(); plt.show()
```

```
Pringles Can slip prevalence (windows): 2.70%  (n = 8,624, slips =
233)
=== PRINGLES CAN: tau = 0.50 vs tau = 0.35 ===
```

Model	Recall @ tau=0.35	Recall @ tau=0.50
Logistic Regression (Approach A)	0.948	0.773
Logistic Regression (Approach B)	0.034	0.034
KNN (Approach A)	0.764	0.549
KNN (Approach B)	0.116	0.064
Random Forest (Approach A)	0.721	0.455
Random Forest (Approach B)	0.017	0.000
LightGBM (Approach A)	0.614	0.438
LightGBM (Approach B)	0.009	0.000
CatBoost (Approach A)	0.609	0.421
CatBoost (Approach B)	0.000	0.000

```
                                  Precision @ tau=0.35  Precision @
tau=0.50  \
Model

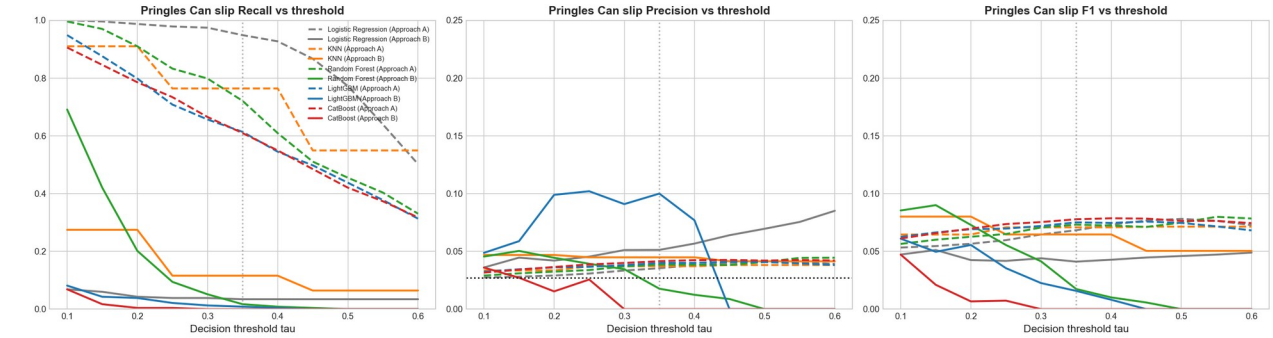
Logistic Regression (Approach A)                 0.035
0.041
Logistic Regression (Approach B)                 0.051
0.070
KNN (Approach A)                                 0.037
0.038
KNN (Approach B)                                 0.045
0.041
Random Forest (Approach A)                       0.039
0.041
Random Forest (Approach B)                       0.018
0.000
LightGBM (Approach A)                            0.040
0.041
LightGBM (Approach B)                            0.100
0.000
CatBoost (Approach A)                            0.042
0.042
CatBoost (Approach B)                            0.000
0.000

                                  FalseAlarms @ tau=0.35  \
Model
Logistic Regression (Approach A)                  6018.0
Logistic Regression (Approach B)                   148.0
KNN (Approach A)                                  4616.0
KNN (Approach B)                                   575.0
Random Forest (Approach A)                        4193.0
Random Forest (Approach B)                         222.0
LightGBM (Approach A)                             3432.0
LightGBM (Approach B)                               18.0
CatBoost (Approach A)                             3276.0
CatBoost (Approach B)                               15.0

                                  FalseAlarms @ tau=0.50  \
Model
Logistic Regression (Approach A)                  4197.0
Logistic Regression (Approach B)                   107.0
KNN (Approach A)                                  3225.0
KNN (Approach B)                                   347.0
Random Forest (Approach A)                        2492.0
Random Forest (Approach B)                          48.0
LightGBM (Approach A)                             2399.0
LightGBM (Approach B)                                0.0
CatBoost (Approach A)                             2235.0
```

```
CatBoost (Approach B)                                    5.0

                                    Precision lift @0.35 (x chance)
Model
Logistic Regression (Approach A)                            1.311
Logistic Regression (Approach B)                            1.898
KNN (Approach A)                                            1.374
KNN (Approach B)                                            1.660
Random Forest (Approach A)                                  1.426
Random Forest (Approach B)                                  0.655
LightGBM (Approach A)                                       1.481
LightGBM (Approach B)                                       3.701
CatBoost (Approach A)                                       1.538
CatBoost (Approach B)                                       0.000
```

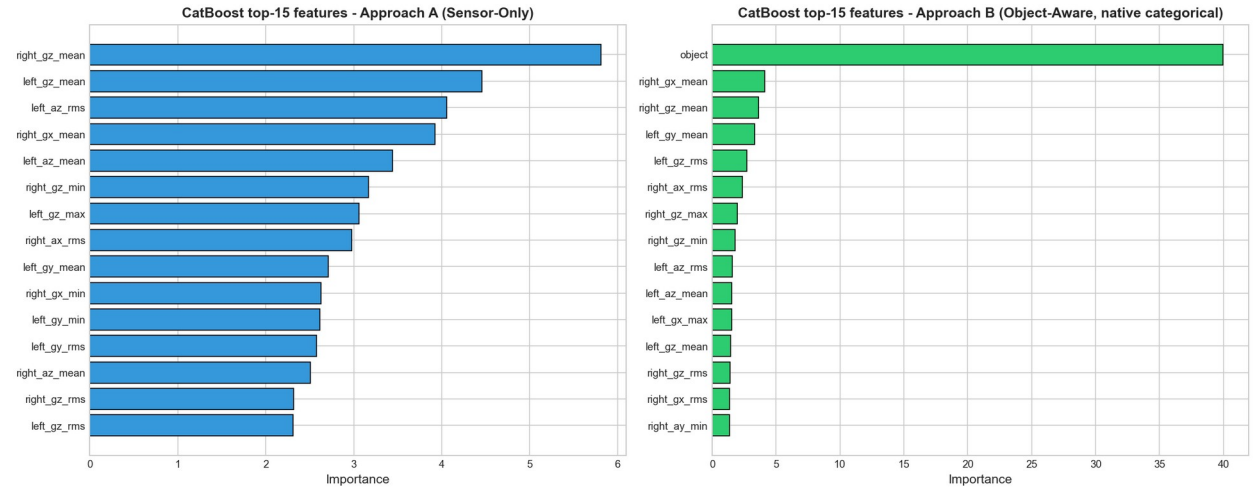


13. Model Interpretability: CatBoost Feature Importance

CatBoost importances (prediction-values change, normalised to 100 and averaged over the five training folds) are shown for the sensor-only and object-aware models. This indicates which engineered statistics the model relies on; it is descriptive and does not by itself establish a physical mechanism. In Approach B, the importance given to the `object` feature shows how much of the gain comes from knowing the object rather than from the IMU signal.

```
# Section 13: CatBoost feature importance (mean over training folds)
cat_imp.to_csv(os.path.join(OUTPUT_DIR,
"catboost_feature_importance.csv"), index=False)
fig, axes = plt.subplots(1, 2, figsize=(15, 6))
for ax, col, ttl, colr in zip(axes, ["Approach_A", "Approach_B"],
["Approach A (Sensor-Only)", "Approach B (Object-Aware, native
categorical)"], ["#3498db", "#2ecc71"]):
    top = cat_imp.dropna(subset=[col]).nlargest(15, col).iloc[::-1]
    ax.barh(top["Feature"], top[col], color=colr, edgecolor="black",
linewidth=0.8)
    ax.set_title(f"CatBoost top-15 features - {ttl}",
fontweight="bold"); ax.set_xlabel("Importance")
plt.tight_layout(); plt.show()
```

```
print(f"Importance share of the 'object' feature in Approach B:
{cat_imp.loc[cat_imp['Feature'] == 'object',
'Approach_B'].iloc[0]:.2f}% of total")
print("Top-5 sensor-only features:", ", ".join(cat_imp.nlargest(5,
"Approach_A")["Feature"]))
```




```
Importance share of the 'object' feature in Approach B: 39.98% of
total
Top-5 sensor-only features: right_gz_mean, left_gz_mean, left_az_rms,
right_gx_mean, left_az_mean
```

---

14. Sensor-Noise Stress Test (Simulated Jitter)

Real actuators add vibration and electronic noise to IMU readings. To get a first indication of sensitivity, zero-mean Gaussian noise with standard deviation $\sigma = p \times (\text{feature std})$, $p \in \{0, 2, 5, 8, 10, 15\}\%$, is added to the **test features only**; models are trained on clean data. Results are averaged over 5 noise draws (fixed seed).

Protocol and limits (read the numbers with these in mind):

- Sensor-only (Approach A) features; LightGBM, Random Forest and CatBoost.
- A **single chronological 80/20 hold-out** (last 20% of rows), not the 5-fold grouped CV, so the clean baseline differs from the cross-validated figures.
- Noise is added at the feature level, not to raw signals.

A steady accuracy drop with noise is evidence of *sensitivity*, not of robustness to motor vibration.

```
# Section 14: noise stress test (trained here, sensor-only features,
single chronological hold-out)
split_idx = int(len(df_features) * 0.8)
overlap = set(df_features["trial_id"].iloc[:split_idx]) &
```

```
set(df_features["trial_id"].iloc[split_idx:])
print(f"Hold-out split check: {len(overlap)} trial(s) straddle the
80/20 cut (rows are ordered by trial).")

Xtr, Xte, ytr, yte = X_A[:split_idx], X_A[split_idx:], y[:split_idx],
y[split_idx:]
noise_models = {
    "LightGBM": LGBMClassifier(n_estimators=100, learning_rate=0.08,
num_leaves=31, random_state=42, n_jobs=-1, verbose=-1),
    "Random Forest": RandomForestClassifier(n_estimators=100,
max_depth=16, random_state=42, n_jobs=-1),
    "CatBoost": CatBoostClassifier(iterations=300, learning_rate=0.08,
depth=6, random_seed=42, verbose=0),
}
for mdl in noise_models.values(): mdl.fit(Xtr, ytr)

feat_std = Xte.std(axis=0)
rng = np.random.default_rng(42)
levels = [0.0, 0.02, 0.05, 0.08, 0.10, 0.15]
rows = []
for nl in levels:
    row = {"Noise (% of feature std)": int(nl * 100)}
    for name, mdl in noise_models.items():
        accs, f1s = [], []
        for _ in range(1 if nl == 0 else 5):
            Xn = Xte + rng.normal(0, nl * feat_std, Xte.shape) if nl >
0 else Xte
            p = mdl.predict(Xn)
            accs.append(accuracy_score(yte, p));
f1s.append(f1_score(yte, p))
        row[f"{name} Acc %"] = np.mean(accs) * 100; row[f"{name} F1
%"] = np.mean(f1s) * 100
    rows.append(row)
noise = pd.DataFrame(rows)
noise.to_csv(os.path.join(OUTPUT_DIR, "sensor_noise_robustness.csv"),
index=False)
display(noise.round(1))
for name in noise_models:
    a0, a1 = noise[f"{name} Acc %"].iloc[0], noise[f"{name} Acc
%"].iloc[-1]
    print(f"{name:<14s}: {a0:.1f}% -> {a1:.1f}% accuracy at 15% noise
({a1 - a0:+.1f} pts)")

fig, ax = plt.subplots(figsize=(9, 5))
for name, mk in zip(noise_models, ["o", "^", "s"]):
    ax.plot(noise["Noise (% of feature std)"], noise[f"{name} Acc %"],
marker=mk, lw=2, label=f"{name} accuracy")
ax.set_xlabel("Gaussian noise (% of feature std)", fontweight="bold");
ax.set_ylabel("Accuracy (%)", fontweight="bold")
ax.set_title("Degradation under simulated feature noise (sensor-only,
```

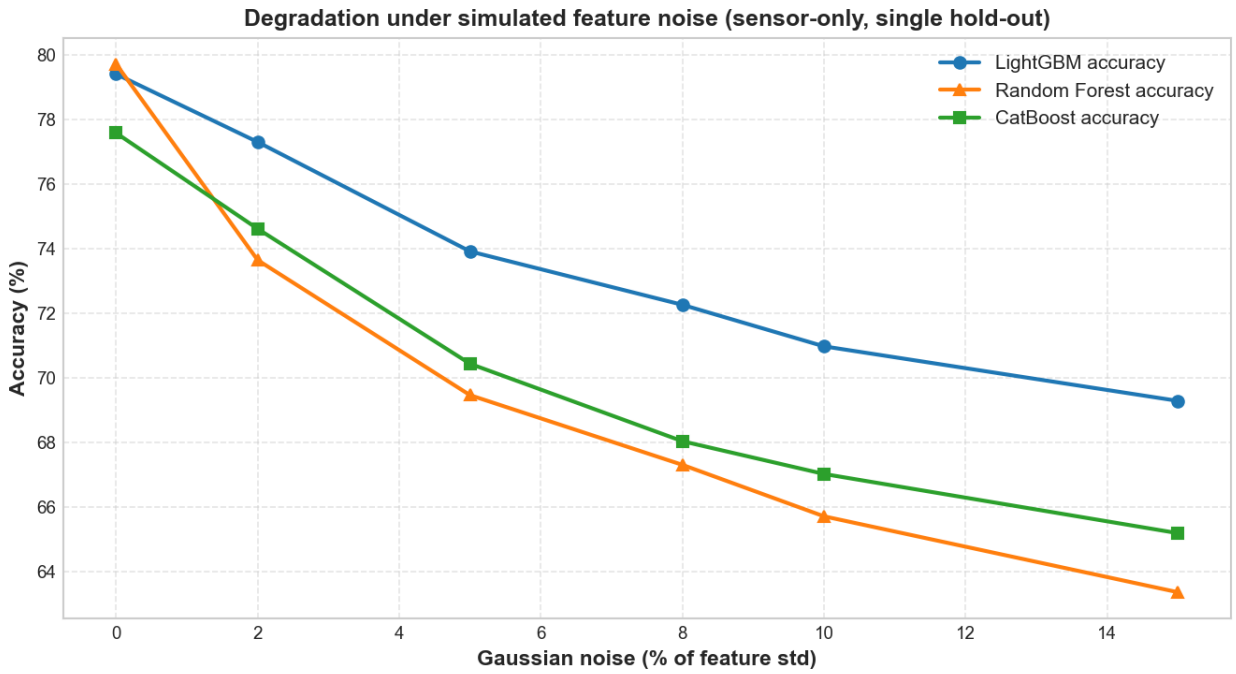
```
single hold-out)", fontweight="bold")
ax.grid(True, ls="--", alpha=0.5); ax.legend()
plt.tight_layout(); plt.show()
```

```
Hold-out split check: 1 trial(s) straddle the 80/20 cut (rows are
ordered by trial).
```

```
   Noise (% of feature std)  LightGBM Acc %  LightGBM F1 %  \
0                         0            79.4           79.7
1                         2            77.3           77.2
2                         5            73.9           73.1
3                         8            72.3           71.1
4                        10            71.0           69.7
5                        15            69.3           68.3

   Random Forest Acc %  Random Forest F1 %  CatBoost Acc %  CatBoost
F1 %
0                 79.7                79.9            77.6
77.7
1                 73.7                72.1            74.6
73.8
2                 69.5                66.1            70.4
68.1
3                 67.3                62.9            68.0
64.6
4                 65.7                60.5            67.0
63.2
5                 63.4                57.4            65.2
60.8
```

```
LightGBM      : 79.4% -> 69.3% accuracy at 15% noise (-10.1 pts)
Random Forest : 79.7% -> 63.4% accuracy at 15% noise (-16.4 pts)
CatBoost      : 77.6% -> 65.2% accuracy at 15% noise (-12.4 pts)
```



---

15. Effect of Object Identity: Approach B minus Approach A (all five algorithms)

$$\Delta\,\text{Metric} = \text{Metric}_{\text{Approach B}} - \text{Metric}_{\text{Approach A}}$$

As noted in Section 9, grouping is by trial, so these gains reflect performance with a *known* object, not generalisation to new objects.

```
# Section 15: object-identity deltas for all five algorithms
metrics = ["Accuracy", "Precision", "Recall", "F1", "ROC_AUC", "FPR"]
rows = []
for m in MODEL_ORDER:
    a = df_comp[(df_comp["Model"] == m) & (df_comp["Approach"] ==
"Approach A")].iloc[0]
    b = df_comp[(df_comp["Model"] == m) & (df_comp["Approach"] ==
"Approach B")].iloc[0]
    rows.append({"Model": m, **{f"Delta_{k}": b[k] - a[k] for k in
metrics}})
df_deltas = pd.DataFrame(rows)
df_deltas.to_csv(os.path.join(OUTPUT_DIR,
"object_identity_metric_deltas.csv"), index=False)
print("=== OBJECT-IDENTITY METRIC DELTAS (Approach B minus Approach A,
5 algorithms) ===")
display(df_deltas.round(4))

fig, ax = plt.subplots(figsize=(10, 4.5))
```

```
dp = df_deltas.set_index("Model")[["Delta_Accuracy", "Delta_F1",
"Delta_ROC_AUC"]]
dp.columns = ["Accuracy", "F1", "ROC AUC"]
dp.plot(kind="bar", ax=ax, colormap="viridis", edgecolor="black",
linewidth=1.0)
ax.set_ylabel("Absolute change (B - A)", fontweight="bold")
ax.set_title("Gain from knowing the object identity (Approach B vs
Approach A)", fontweight="bold")
ax.set_xticklabels(ax.get_xticklabels(), rotation=0)
plt.tight_layout(); plt.show()

=== OBJECT-IDENTITY METRIC DELTAS (Approach B minus Approach A, 5
algorithms) ===

                  Model  Delta_Accuracy  Delta_Precision  Delta_Recall
\
0  Logistic Regression          0.1694           0.1593        0.2033

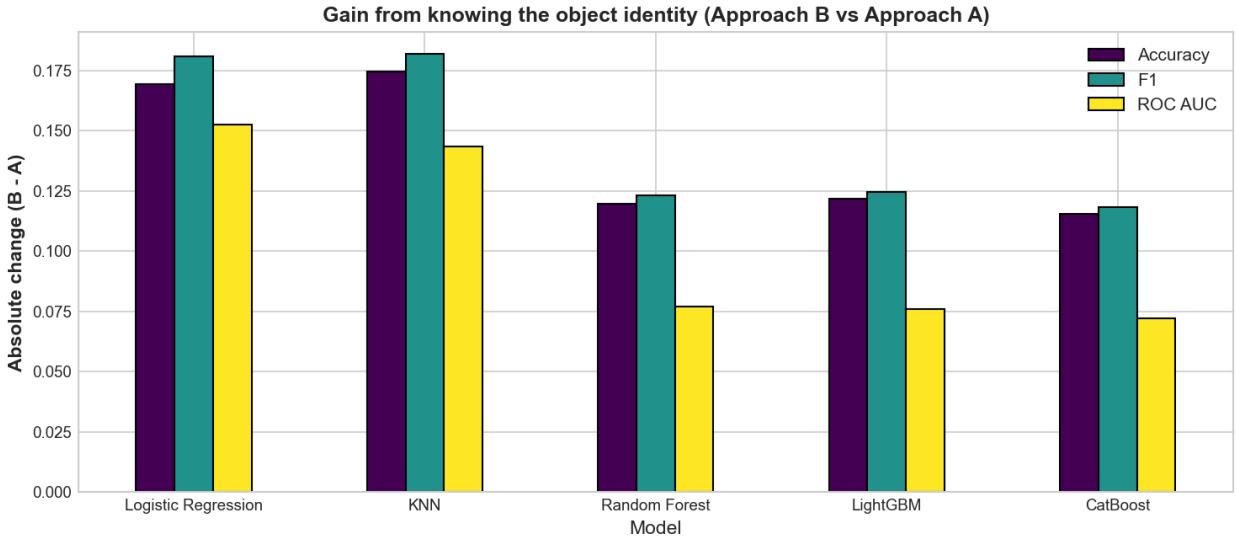
1                  KNN          0.1747           0.1761        0.1875

2        Random Forest          0.1198           0.1311        0.1151

3             LightGBM          0.1218           0.1341        0.1147

4             CatBoost          0.1155           0.1273        0.1090


   Delta_F1  Delta_ROC_AUC  Delta_FPR
0    0.1810         0.1525    -0.1382
1    0.1817         0.1435    -0.1628
2    0.1232         0.0771    -0.1241
3    0.1246         0.0758    -0.1283
4    0.1183         0.0720    -0.1214
```



16. Conclusions, Literature Context & Limitations

16.1 Context from the Reference Paper (Cravetz et al., 2025)

The reference study (*Frontiers in Robotics and AI*, 2025) trains a **convolutional neural network (CNN)** on continuous raw fingertip IMU signals and reports results at two specific operating thresholds:

- **Balanced Operating Threshold**: reports an **F1-score of 0.70** (at a decision threshold of 0.55).
- **Conservative High-Precision Threshold**: reports **93.1% precision and 25.2% recall** (at a threshold of 0.98).

The present classical machine learning pipeline evaluates hand-crafted 84 summary statistics computed over 250 ms sliding windows under 5-fold trial-grouped cross-validation (,022$ windows, 192 clean physical trials). Because window duration, kinematic thresholding (.06\ text{ m/s}$), sample aggregation rules, and validation protocols differ, the figures are **not directly comparable**, and no claim of direct model superiority over the published deep learning architecture is made.

16.2 Key Empirical Findings Supported by the Pipeline

1. **Classical Machine Learning Baseline**: Under strictly leakage-free 5-fold GroupKFold cross-validation:
   - **Approach A (Sensor-Only)**: The best model is CatBoost, reaching **84.55% Accuracy, 0.8411 F1, and 0.9141 ROC-AUC** (LightGBM reaches 83.46% / 0.8301 F1; Random Forest 82.81% / 0.8225 F1).
   - **Approach B (Object-Aware)**: Conditioning on known object identity improves accuracy across all algorithms, with CatBoost reaching **96.09% Accuracy, 0.9594 F1, and 0.9861 ROC-AUC**.
2. **Contextual Gain from Object Identity**: As demonstrated in Section 15, knowing the object geometry and compliance provides an absolute gain of $+11.5\,\%$ to $+17.5\,\%$ accuracy across all 5 algorithms. Note that cross-validation is grouped by physical trial, meaning objects are shared across folds; this reflects performance with a *known* object, not zero-shot generalization to novel, unencountered objects.
3. **The Pringles Can Rare-Slip Challenge**: On the Pringles Can (only 2.7% slip prevalence), object-aware tree models collapse to .0%$ slip recall at default $\tau = 0.50$ due to severe class imbalance. Lowering the decision threshold to $\tau = 0.35$ raises sensor-only recall to .1% - 94.8%$ (tree models .1% - 72.1% $¿, but precision remains low at .5\,\% - 4.2\,\%$ (approximately .3\times - 1.5\times$ chance), yielding thousands of false alarms. Threshold tuning exposes this fundamental precision-recall trade-off; it does not represent a fully validated operational solution.
4. **Sensor Noise Sensitivity**: Simulating actuator jitter on sensor-only test features causes measurable performance degradation: LightGBM drops from 79.4% to 69.3%, Random Forest drops from 79.7% to 63.4%, and CatBoost drops from 77.6% to 65.2% under 15% Gaussian feature noise.

16.3 Project Limitations

- All experiments originate from a single compliant gripper setup and 5 specific objects.
- Labels are assigned via a 30% slip-fraction rule over a kinematic contact speed threshold (.06\text{ m/s}$).
- The sensor-noise stress test perturbs extracted feature statistics on a single chronological holdout rather than injecting physical electrical noise into raw continuous waveform telemetry.

```
# Section 16: run manifest (all figures are taken from the tables
computed above)
best = df_comp.loc[df_comp["Accuracy"].idxmax()]
best_A = df_comp[df_comp["Approach"] == "Approach A"].loc[lambda d:
d["Accuracy"].idxmax()]
p35 = th[th["Threshold"] == 0.35]
manifest = {
    "project_name": "SLID-CH tactile slip detection - classical ML
pipeline",
    "dataset": "SLID-CH (195 trials, 192 evaluated after excluding 3
sensor-fault trials)",
    "windows_evaluated": int(len(df_features)),
    "features_extracted": 84,
    "cross_validation": "5-fold GroupKFold by trial_id (no trial
leakage; objects are shared across folds)",
    "training_protocol": "fixed hyper-parameters, no tuning or early
stopping on held-out folds",
    "algorithms": MODEL_ORDER,
    "best_sensor_only": {"model": best_A["Model"], "accuracy":
round(float(best_A["Accuracy"]), 4), "f1": round(float(best_A["F1"]),
4)},
    "best_object_aware": {"model": f"{best['Model']} -
{best['Approach']}", "accuracy": round(float(best["Accuracy"]), 4),
"f1": round(float(best["F1"]), 4)},
    "pringles_threshold_0.35": {r["Model"]: {"recall":
round(float(r["Recall"]), 3), "precision":
round(float(r["Precision"]), 3)} for _, r in p35.iterrows()},
    "pringles_note": "Not a validated operating point: see Section
12.",
    "noise_test": {"protocol": "single chronological 80/20 hold-out,
sensor-only features, mean of 5 noise draws",
                   **{f"{n}_acc_at_15pct": round(float(noise[f'{n} Acc
%'].iloc[-1]), 1) for n in noise_models}},
    "literature_note": "Cravetz et al. (2025) report a CNN with F1
0.70 (balanced threshold) and 93.1% precision / 25.2% recall
(conservative threshold); protocols differ, so no direct comparison is
claimed.",
}
manifest_path = os.path.join(OUTPUT_DIR, "run_manifest.json")
with open(manifest_path, "w") as f:
    json.dump(manifest, f, indent=2)
```

```python
print("=== PROJECT EXECUTION COMPLETE ===")
print(f"Run manifest saved to: {manifest_path}")
print(json.dumps(manifest["best_sensor_only"]),
json.dumps(manifest["best_object_aware"]))
```

```
=== PROJECT EXECUTION COMPLETE ===
Run manifest saved to: C:\Vishal\Datascience\outputs_master\
run_manifest.json
{"model": "CatBoost", "accuracy": 0.8455, "f1": 0.8411} {"model":
"CatBoost - Approach B", "accuracy": 0.9609, "f1": 0.9594}
```